NAME: Ebenezer DABIRE

ROLL NO: 2025115168

1. Write a C program to store **n elements in an array using dynamic memory allocation** and find the **largest number**.

Code:

```
#include <stdio.h>

#include <stdlib.h>


int main()

{

   int n, i, largest;

   int *arr;


   printf("Enter the number of elements: ");

   scanf("%d", &n);


   // Dynamic memory allocation

   arr = (int *)malloc(n * sizeof(int));


   if (arr == NULL)

   {

      printf("Memory allocation failed!\n");

      return 1;

   }


   // Read elements

   printf("Enter %d elements:\n", n);

   for (i = 0; i < n; i++)

   {

      scanf("%d", &arr[i]);

   }
```

```
    // Assume first element is largest
    largest = arr[0];

    // Find largest element
    for (i = 1; i < n; i++)
    {
        if (arr[i] > largest)
        {
            largest = arr[i];
        }
    }

    printf("Largest number = %d\n", largest);

    // Free allocated memory
    free(arr);

    return 0;
}
```

Output:

```
Enter the number of elements: 5
Enter 5 elements:
12 45 7 89 34
Largest number = 89
```

2. Print an array in **reverse order using pointers**.

Code:

```
#include <stdio.h>

int main()
{
```

```
    int n, i;
    int arr[100];
    int *ptr;

    printf("Enter the number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    // Pointer points to the last element
    ptr = &arr[n - 1];

    printf("Array in reverse order:\n");

    for (i = n - 1; i >= 0; i--)
    {
        printf("%d ", *ptr);
        ptr--;
    }

    return 0;
}
```

Output:

Enter the number of elements: 5

Enter 5 elements:

10 20 30 40 50

Array in reverse order:

50 40 30 20 10

3. Write a C program to store information of **students using structures**. Sort them by **name**.

Code:

```
#include <stdio.h>
#include <string.h>

struct Student
{
   int rollno;
   char name[50];
   float marks;
};

int main()
{
   struct Student s[100], temp;
   int n, i, j;

   printf("Enter the number of students: ");
   scanf("%d", &n);

   // Read student details
   for (i = 0; i < n; i++)
   {
      printf("\nEnter details of student %d:\n", i + 1);

      printf("Roll Number: ");
      scanf("%d", &s[i].rollno);

      printf("Name: ");
```

```
    scanf(" %[^\n]", s[i].name);

    printf("Marks: ");
    scanf("%f", &s[i].marks);
}

// Sort students by name
for (i = 0; i < n - 1; i++)
{
    for (j = i + 1; j < n; j++)
    {
        if (strcmp(s[i].name, s[j].name) > 0)
        {
            temp = s[i];
            s[i] = s[j];
            s[j] = temp;
        }
    }
}

// Display sorted student details
printf("\nStudents sorted by name:\n");
printf("Roll No\tName\t\tMarks\n");

for (i = 0; i < n; i++)
{
    printf("%d\t%s\t\t%.2f\n",
           s[i].rollno, s[i].name, s[i].marks);
}

return 0;
```

```
}
```

Output:

```
Enter the number of students: 3

Enter details of student 1:
Roll Number: 101
Name: Rahul
Marks: 85

Enter details of student 2:
Roll Number: 102
Name: Arun
Marks: 92

Enter details of student 3:
Roll Number: 103
Name: Karthik
Marks: 78

Students sorted by name:
Roll No Name          Marks
102     Arun          92.00
103     Karthik       78.00
101     Rahul         85.00
```

4. Write a C program to **add 'n' distances given in inch–feet system using structures**

Code:

```
#include <stdio.h>

struct Distance
{
   int feet;
```

```
    float inch;
};

int main()
{
    struct Distance d[100], sum = {0, 0};
    int n, i;

    printf("Enter the number of distances: ");
    scanf("%d", &n);

    // Read distances
    for (i = 0; i < n; i++)
    {
        printf("\nEnter distance %d:\n", i + 1);

        printf("Feet: ");
        scanf("%d", &d[i].feet);

        printf("Inches: ");
        scanf("%f", &d[i].inch);

        sum.feet += d[i].feet;
        sum.inch += d[i].inch;
    }

    // Convert inches to feet
    if (sum.inch >= 12)
    {
        sum.feet += (int)(sum.inch / 12);
        sum.inch = (int)sum.inch % 12;
```

```
    }

    printf("\nTotal Distance = %d feet %.2f inches\n",
        sum.feet, sum.inch);

    return 0;
}
```

Output:

```
Enter the number of distances: 3

Enter distance 1:
Feet: 5
Inches: 8

Enter distance 2:
Feet: 3
Inches: 9

Enter distance 3:
Feet: 2
Inches: 7

Total Distance = 12 feet 0.00 inches
```

5. Write a C program to **add two complex numbers by passing structures to a function**.

Code:

```
#include <stdio.h>

struct Complex
{
    float real;
    float imag;
```

```
};

// Function to add two complex numbers
struct Complex add(struct Complex c1, struct Complex c2)
{
   struct Complex result;

   result.real = c1.real + c2.real;
   result.imag = c1.imag + c2.imag;

   return result;
}

int main()
{
   struct Complex c1, c2, sum;

   printf("Enter real and imaginary parts of first complex number: ");
   scanf("%f %f", &c1.real, &c1.imag);

   printf("Enter real and imaginary parts of second complex number: ");
   scanf("%f %f", &c2.real, &c2.imag);

   // Pass structures to function
   sum = add(c1, c2);

   printf("\nSum = %.2f + %.2fi\n", sum.real, sum.imag);

   return 0;
}
```

Output:

Enter real and imaginary parts of first complex number: 3 4

Enter real and imaginary parts of second complex number: 5 2

Sum = 8.00 + 6.00i

6. Write a C program to implement **Inventory Management System using structures**.

Code:

```
#include <stdio.h>
#include <string.h>

struct Product
{
   int id;
   char name[50];
   int quantity;
   float price;
};

int main()
{
   struct Product p[100];
   int n = 0;
   int choice, i, id, found;

   do
   {
      printf("\n===== INVENTORY MANAGEMENT SYSTEM =====\n");
      printf("1. Add Product\n");
      printf("2. Display Products\n");
      printf("3. Search Product\n");
      printf("4. Update Quantity\n");
      printf("5. Exit\n");
```

```
printf("Enter your choice: ");
scanf("%d", &choice);

switch (choice)
{
   case 1:
      printf("\nEnter Product ID: ");
      scanf("%d", &p[n].id);

      printf("Enter Product Name: ");
      scanf(" %[^\n]", p[n].name);

      printf("Enter Quantity: ");
      scanf("%d", &p[n].quantity);

      printf("Enter Price: ");
      scanf("%f", &p[n].price);

      n++;

      printf("Product added successfully!\n");
      break;

   case 2:
      if (n == 0)
      {
         printf("\nNo products available.\n");
      }
      else
      {
```

```
        printf("\n------ PRODUCT LIST ------\n");
        printf("ID\tName\t\tQuantity\tPrice\n");

        for (i = 0; i < n; i++)
        {
            printf("%d\t%s\t\t%d\t\t%.2f\n",
                p[i].id,
                p[i].name,
                p[i].quantity,
                p[i].price);
        }
    }
    break;

case 3:
    printf("\nEnter Product ID to search: ");
    scanf("%d", &id);

    found = 0;

    for (i = 0; i < n; i++)
    {
        if (p[i].id == id)
        {
            printf("\nProduct Found!\n");
            printf("ID      : %d\n", p[i].id);
            printf("Name    : %s\n", p[i].name);
            printf("Quantity : %d\n", p[i].quantity);
            printf("Price   : %.2f\n", p[i].price);

            found = 1;
```

```
            break;
        }
    }

    if (found == 0)
    {
        printf("Product not found.\n");
    }
    break;

case 4:
    printf("\nEnter Product ID to update: ");
    scanf("%d", &id);

    found = 0;

    for (i = 0; i < n; i++)
    {
        if (p[i].id == id)
        {
            printf("Enter new quantity: ");
            scanf("%d", &p[i].quantity);

            printf("Quantity updated successfully!\n");

            found = 1;
            break;
        }
    }

    if (found == 0)
```

```
            {
                printf("Product not found.\n");
            }
            break;

        case 5:
            printf("\nExiting Inventory Management System...\n");
            break;

        default:
            printf("\nInvalid choice!\n");
    }

  } while (choice != 5);

  return 0;
}
```

Output:

```
===== INVENTORY MANAGEMENT SYSTEM =====
1. Add Product
2. Display Products
3. Search Product
4. Update Quantity
5. Exit

Enter your choice: 1

Enter Product ID: 101
Enter Product Name: Laptop
Enter Quantity: 10
Enter Price: 55000
```

Product added successfully!

Enter your choice: 1

Enter Product ID: 102

Enter Product Name: Mouse

Enter Quantity: 25

Enter Price: 500

Product added successfully!

Enter your choice: 2

------ PRODUCT LIST ------

ID Name Quantity Price

101 Laptop 10 55000.00

102 Mouse 25 500.00

7. Write a C program for **decimal to binary, octal and hexadecimal conversion**

Code:

```
#include <stdio.h>

int main()
{
   int decimal;

   printf("Enter a decimal number: ");
   scanf("%d", &decimal);

   // Binary
   printf("Binary     : ");
   int n = decimal;
```

```
    int binary[32], i = 0;

    if (n == 0)
    {
        printf("0");
    }
    else
    {
        while (n > 0)
        {
            binary[i] = n % 2;
            n = n / 2;
            i++;
        }

        while (i > 0)
        {
            i--;
            printf("%d", binary[i]);
        }
    }

    // Octal
    printf("\nOctal      : %o", decimal);

    // Hexadecimal
    printf("\nHexadecimal : %X", decimal);

    return 0;
}
```

Output:

Enter a decimal number: 25

Binary : 11001

Octal : 31

Hexadecimal : 19

8. Write a C program to calculate the **difference between two time periods**.

Code:

```
#include <stdio.h>

struct Time
{
   int hours;
   int minutes;
   int seconds;
};

int main()
{
   struct Time t1, t2, diff;

   printf("Enter first time period (hours minutes seconds): ");
   scanf("%d %d %d", &t1.hours, &t1.minutes, &t1.seconds);

   printf("Enter second time period (hours minutes seconds): ");
   scanf("%d %d %d", &t2.hours, &t2.minutes, &t2.seconds);

   // Make sure t1 is the larger time
   if (t1.hours < t2.hours ||
      (t1.hours == t2.hours && t1.minutes < t2.minutes) ||
      (t1.hours == t2.hours && t1.minutes == t2.minutes &&
       t1.seconds < t2.seconds))
```

```
{
   struct Time temp = t1;
   t1 = t2;
   t2 = temp;
}

// Subtract seconds
if (t1.seconds < t2.seconds)
{
   t1.seconds += 60;
   t1.minutes--;
}

diff.seconds = t1.seconds - t2.seconds;

// Subtract minutes
if (t1.minutes < t2.minutes)
{
   t1.minutes += 60;
   t1.hours--;
}

diff.minutes = t1.minutes - t2.minutes;

// Subtract hours
diff.hours = t1.hours - t2.hours;

printf("\nDifference = %d hours %d minutes %d seconds\n",
     diff.hours, diff.minutes, diff.seconds);

return 0;
```

```
}
```

Output:

Enter first time period (hours minutes seconds): 8 45 30

Enter second time period (hours minutes seconds): 5 30 45

Difference = 3 hours 14 minutes 45 seconds

9. Write a C program to develop an **Employee Management System using structures**

Code:

```
#include <stdio.h>
#include <string.h>

struct Employee
{
   int id;
   char name[50];
   char department[50];
   float salary;
};

int main()
{
   struct Employee emp[100];
   int n = 0;
   int choice, i, id, found;

   do
   {
      printf("\n===== EMPLOYEE MANAGEMENT SYSTEM =====\n");
      printf("1. Add Employee\n");
      printf("2. Display Employees\n");
      printf("3. Search Employee\n");
```

```
printf("4. Update Salary\n");
printf("5. Exit\n");

printf("Enter your choice: ");
scanf("%d", &choice);

switch (choice)
{
   case 1:
      printf("\nEnter Employee ID: ");
      scanf("%d", &emp[n].id);

      printf("Enter Employee Name: ");
      scanf(" %[^\n]", emp[n].name);

      printf("Enter Department: ");
      scanf(" %[^\n]", emp[n].department);

      printf("Enter Salary: ");
      scanf("%f", &emp[n].salary);

      n++;

      printf("Employee added successfully!\n");
      break;

   case 2:
      if (n == 0)
      {
         printf("\nNo employees available.\n");
      }
```

```
    else
    {
        printf("\n---------- EMPLOYEE DETAILS ----------\n");

        for (i = 0; i < n; i++)
        {
            printf("\nEmployee %d\n", i + 1);
            printf("ID       : %d\n", emp[i].id);
            printf("Name     : %s\n", emp[i].name);
            printf("Department : %s\n", emp[i].department);
            printf("Salary   : %.2f\n", emp[i].salary);
        }
    }
    break;

case 3:
    printf("\nEnter Employee ID to search: ");
    scanf("%d", &id);

    found = 0;

    for (i = 0; i < n; i++)
    {
        if (emp[i].id == id)
        {
            printf("\nEmployee Found!\n");
            printf("ID       : %d\n", emp[i].id);
            printf("Name     : %s\n", emp[i].name);
            printf("Department : %s\n", emp[i].department);
            printf("Salary   : %.2f\n", emp[i].salary);
```

```
            found = 1;
            break;
        }
    }

    if (found == 0)
    {
        printf("Employee not found.\n");
    }
    break;

case 4:
    printf("\nEnter Employee ID to update salary: ");
    scanf("%d", &id);

    found = 0;

    for (i = 0; i < n; i++)
    {
        if (emp[i].id == id)
        {
            printf("Enter new salary: ");
            scanf("%f", &emp[i].salary);

            printf("Salary updated successfully!\n");

            found = 1;
            break;
        }
    }
```

```
            if (found == 0)
            {
               printf("Employee not found.\n");
            }
            break;

        case 5:
            printf("\nExiting Employee Management System...\n");
            break;

        default:
            printf("\nInvalid choice!\n");
      }

   } while (choice != 5);

   return 0;
}
```

Output:

```
===== EMPLOYEE MANAGEMENT SYSTEM =====
1. Add Employee
2. Display Employees
3. Search Employee
4. Update Salary
5. Exit

Enter your choice: 1

Enter Employee ID: 101
Enter Employee Name: Arun
Enter Department: IT
```

Enter Salary: 45000

Employee added successfully!

Enter your choice: 2

---------- EMPLOYEE DETAILS ----------

Employee 1

ID : 101

Name : Arun

Department : IT

Salary : 45000.00

10. Write a C program to calculate **area and perimeter of a rectangle and triangle using function pointers**.

Code:

```
#include <stdio.h>

#include <math.h>


// Rectangle functions

float rectangleArea(float length, float breadth)

{

  return length * breadth;

}


float rectanglePerimeter(float length, float breadth)

{

  return 2 * (length + breadth);

}


// Triangle functions
```

```
float triangleArea(float base, float height)
{
   return 0.5 * base * height;
}

float trianglePerimeter(float a, float b, float c)
{
   return a + b + c;
}

int main()
{
   float length, breadth;
   float base, height, a, b, c;

   // Function pointers
   float (*area)(float, float);
   float (*perimeter)(float, float);

   // Rectangle
   printf("Enter length and breadth of rectangle: ");
   scanf("%f %f", &length, &breadth);

   area = rectangleArea;
   perimeter = rectanglePerimeter;

   printf("\nRectangle Area = %.2f\n", area(length, breadth));
   printf("Rectangle Perimeter = %.2f\n", perimeter(length, breadth));

   // Triangle
   printf("\nEnter base and height of triangle: ");
```

```
    scanf("%f %f", &base, &height);

    printf("Enter three sides of triangle: ");
    scanf("%f %f %f", &a, &b, &c);

    area = triangleArea;

    printf("\nTriangle Area = %.2f\n", area(base, height));

    // Triangle perimeter needs three arguments,
    // so use a separate function pointer.
    float (*trianglePeri)(float, float, float);
    trianglePeri = trianglePerimeter;

    printf("Triangle Perimeter = %.2f\n",
        trianglePeri(a, b, c));

    return 0;
}
```

Output:

```
Enter length and breadth of rectangle: 10 5

Rectangle Area = 50.00
Rectangle Perimeter = 30.00

Enter base and height of triangle: 8 6
Enter three sides of triangle: 8 6 10

Triangle Area = 24.00
Triangle Perimeter = 24.00
```